

A Report on

RTL to GDSII Implementation of 8-bit Arithmetic and Logic Unit (ALU)

Using Synopsys EDA Tool Flow

Submitted in partial fulfilment of the requirements for the
RTL-to-GDSII Workshop by,

Aditya S. Joshi

Workshop Conducted by:

VLSI Expert Pvt. Ltd. and SemiconTantra Foundation

In collaboration with Synopsys Inc.

Workshop Duration: 4 Days

Technology Node: SAED 32nm

Tools: VCS, Verdi, DC, ICC2, PrimeTime.

Contents

1. Introduction
2. Objectives
3. Overview of RTL-to-GDSII Flow
4. ALU Architecture and Design Specification
5. RTL Design
6. Functional Verification (VCS & Verdi)
7. Synthesis using Design Compiler
8. Constraint Definition (SDC)
9. Floorplanning (ICC2)
10. Power Planning
11. Placement
12. Clock Tree Synthesis
13. Routing
14. Static Timing Analysis using PrimeTime
15. Conclusion

1. Introduction

The increasing complexity of modern digital systems demands a structured and tool-driven ASIC design methodology. The RTL-to-GDSII flow represents the complete transformation of a high-level hardware description into a manufacturable silicon layout.

This project focuses on the complete implementation of an 8-bit Arithmetic Logic Unit (ALU) using industry-standard Synopsys EDA tools. The design flow includes:

- RTL development in Verilog
- Functional verification
- Logic synthesis
- Physical design implementation
- Clock Tree Synthesis
- Routing
- Static Timing Analysis
- Signoff verification

The objective was not only to achieve a functionally correct design, but also to ensure timing closure, physical feasibility, and design rule compliance.

2. Objectives

The primary objectives of this project were:

1. To design and implement an 8-bit ALU in Verilog HDL.
2. To verify the functional correctness using simulation tools.
3. To perform logic synthesis with timing constraints.
4. To execute complete physical design flow using ICC2.
5. To achieve timing closure using PrimeTime.
6. To generate a DRC-clean and timing-clean routed layout.
7. To understand industrial ASIC implementation methodology.

3. Overview of RTL-to-GDSII Flow

The complete ASIC design flow followed in this project is illustrated below:

RTL → Simulation → Synthesis → Floorplanning → Power Planning → Placement → CTS → Routing → STA → GDSII

Each stage transforms the abstraction level of the design:

Stage	Output
RTL	Behavioral Description

Synthesis	Gate-level Netlist
Floorplan	Physical Core Structure
Placement	Cell Locations
CTS	Clock Distribution Network
Routing	Metal Interconnections
STA	Timing Validation
GDSII	Final Layout Database

This structured flow ensures manufacturability and performance compliance.

4. ALU Architecture and Design Specification

4.1. Introduction

The Arithmetic and Logic Unit (ALU) is a fundamental computational block in digital systems and microprocessors. It performs arithmetic operations such as addition and subtraction, as well as logical operations including AND, OR, and XOR. The ALU executes operations based on control signals and generates both data results and status flags.

4.2. Design Overview

The implemented 8-bit ALU is designed as a fully synchronous pipelined module. The architecture consists of input registers, a combinational ALU core, and output registers.

The top-level module (`alu_top`) acts as a wrapper around the combinational `alu_core`. All inputs and outputs are registered to ensure stable timing behavior and improved compatibility with ASIC implementation flow.

4.3. Interface Specification

Inputs

- `clk` - Positive-edge triggered system clock
- `rst` - Asynchronous active-high reset
- `A[7:0]` - First 8-bit operand
- `B[7:0]` - Second 8-bit operand
- `opcode[2:0]` - 3-bit operation select signal

Outputs

- `result[7:0]` - 8-bit computed result
- `zero` - Flag indicating result equals zero
- `carry` - Carry-out flag from arithmetic operations

4.4. Architectural Description

The ALU architecture is divided into three major stages:

1. Input Register Stage

The operands (A, B) and control signal (opcode) are registered on the rising edge of the clock.

This stage:

- Synchronizes external inputs
- Reduces direct combinational dependency from inputs
- Improves timing predictability

Upon reset, all input registers are cleared to zero.

2. Combinational ALU Core

The registered inputs are passed to the `alu_core` module, which is purely combinational.

This block performs arithmetic and logical operations based on the opcode and produces:

- `alu_result` (8-bit result)
- `alu_carry` (carry output)

Since this block contains combinational logic, it forms the primary timing path of the design.

3. Output Register Stage

The outputs of the combinational core are captured in registers on the rising edge of the clock.

- `result` stores the computed value
- `zero` is asserted when the result equals zero
- `carry` captures the carry-out signal

This ensures:

- Glitch-free outputs
- Stable synchronous behaviour
- Accurate static timing analysis

4.5. Pipeline Behavior

Due to the presence of both input and output registers, the design introduces a one-clock-cycle latency between input application and final output availability.

This pipelined structure:

- Breaks long combinational paths
- Improves timing closure
- Enhances physical design feasibility

4.6. Design Characteristics

The implemented ALU follows proper synchronous design methodology with:

- Edge-triggered flip-flops
- Separate sequential and combinational logic
- No inferred latches
- Clear reset behaviour
- ASIC-friendly architecture

Arithmetic operations within the combinational core are expected to contribute to the critical path during synthesis and timing analysis.

5. RTL Design Implementation

The 8-bit ALU was implemented using a modular and hierarchical RTL design approach in Verilog HDL. The design is structured into multiple synthesizable modules to improve readability, maintainability, and synthesis efficiency.

The complete RTL implementation consists of the following source files:

- `alu_top.v` – Top-level wrapper module
- `alu_core.v` – Core ALU control and operation selection
- `adder_8bit.v` – Arithmetic unit (addition/subtraction)
- `logic_unit.v` – Logical operation unit
- `shifter.v` – Shift operation unit

This modular structure enables clear separation between arithmetic, logical, and shift functionalities while ensuring synthesizable and timing-friendly design.

5.1. Top Level Module - `alu_top.v`

The `alu_top` module integrates input registers, the combinational ALU core, and output registers. It ensures synchronous operation with registered inputs and outputs for improved timing behavior.

```
`timescale 1ns/1ps
module alu_top (
    input clk,
    input rst,
    input [7:0] A,
    input [7:0] B,
    input [2:0] opcode,
    output reg [7:0] result,
```

```

        output reg zero,
        output reg carry
    );

    // Registered inputs
    reg [7:0] A_reg, B_reg;
    reg [2:0] opcode_reg;
    // Internal wires
    wire [7:0] alu_result;
    wire alu_carry;
    // Input Registers
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            A_reg <= 0;
            B_reg <= 0;
            opcode_reg <= 0;
        end else begin
            A_reg <= A;
            B_reg <= B;
            opcode_reg <= opcode;
        end
    end
    // Instantiate combinational ALU core
    alu_core core (
        .A(A_reg),
        .B(B_reg),
        .opcode(opcode_reg),
        .result(alu_result),
        .carry(alu_carry)
    );
    // Output Registers
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            result <= 0;
            zero <= 0;
            carry <= 0;
        end else begin
            result <= alu_result;
            zero <= (alu_result == 0);
            carry <= alu_carry;
        end
    end
end
endmodule

```

5.2. ALU Core Module - alu_core.v

The alu_core module acts as the main control block of the ALU. It selects the required operation based on the opcode and connects the arithmetic, logical, and shift units accordingly.

```

`timescale 1ns/1ps
module alu_core (
    input  [7:0] A,
    input  [7:0] B,
    input  [2:0] opcode,
    output reg [7:0] result,
    output carry

```

```

);
wire [7:0] add_result;
wire add_cout;
wire [7:0] logic_result;
wire [7:0] shift_result;
reg cin;
reg [1:0] logic_sel;
reg shift_dir;
adder_8bit U1 (
    .A(A),
    .B(B),
    .cin(cin),
    .sum(add_result),
    .cout(add_cout)
);
logic_unit U2 (
    .A(A),
    .B(B),
    .sel(logic_sel),
    .result(logic_result)
);
shifter U3 (
    .A(A),
    .dir(shift_dir),
    .result(shift_result)
);
always @(*) begin
    cin = 0;
    logic_sel = 2'b00;
    shift_dir = 0;

    case(opcode)
        3'b000: result = add_result;
        3'b001: begin cin = 1; result = add_result; end
        3'b010: begin logic_sel = 2'b00; result = logic_result; end
        3'b011: begin logic_sel = 2'b01; result = logic_result; end
        3'b100: begin logic_sel = 2'b10; result = logic_result; end
        3'b101: begin logic_sel = 2'b11; result = logic_result; end
        3'b110: begin shift_dir = 0; result = shift_result; end
        3'b111: begin shift_dir = 1; result = shift_result; end
    endcase
end
assign carry = add_cout;
endmodule

```

5.3. Arithmetic Unit - adder_8bit.v

```

`timescale 1ns/1ps
module adder_8bit (
    input  [7:0] A,
    input  [7:0] B,
    input      cin,
    output [7:0] sum,
    output      cout
);
assign {cout, sum} = A + B + cin;
endmodule

```

5.4. Logic Unit - logic_unit.v

The logic_unit module performs bitwise logical operations such as AND, OR, and XOR based on control inputs from the ALU core.

```
`timescale 1ns/1ps
module logic_unit (
    input  [7:0] A,
    input  [7:0] B,
    input  [1:0] sel,
    output reg [7:0] result
);
always @(*) begin
    case(sel)
        2'b00: result = A & B;
        2'b01: result = A | B;
        2'b10: result = A ^ B;
        2'b11: result = ~A;
    endcase
end
endmodule
```

5.5. Shifter Unit - shifter.v

The shifter module implements shift operations (logical left or logical right shifts) as required by the opcode.

```
`timescale 1ns/1ps
module shifter (
    input  [7:0] A,
    input          dir,    // 0 = left, 1 = right
    output [7:0] result
);
assign result = (dir == 0) ? (A << 1) : (A >> 1);
endmodule
```

5.6. Design Characteristics

The RTL design follows proper synchronous design methodology with:

- Registered input and output stages
- Clear separation of combinational and sequential logic
- No inferred latches
- Deterministic reset behavior
- Single clock domain operation

The combinational path within the ALU core (particularly arithmetic operations) forms the critical timing path and is later analyzed during synthesis and static timing analysis.

6. Functional Verification using VCS and Verdi

6.1. Verification Overview

Functional verification was performed to validate the correctness of the 8-bit ALU design before proceeding to synthesis and physical implementation. The simulation was carried out using Synopsys VCS, and waveform analysis was performed using Verdi.

The objective of this stage was to ensure that all arithmetic, logical, and shift operations produce correct outputs for different opcode and operand combinations.

6.2. Testbench Strategy

A dedicated testbench was developed to verify the ALU functionality. The testbench performed the following tasks:

- Clock generation
- Reset initialization
- Application of multiple operand combinations
- Opcode variation to test all supported operations
- Monitoring of result, zero flag, and carry flag

The design includes registered inputs and outputs; therefore, a one-clock-cycle latency was considered during output validation.

6.3. Simulation Commands

The RTL files were compiled and simulated using the following commands:

```
vcs -full64 rtl/ALU/*.v tb/ALU_tb.v -debug_access+all
```

```
./simv
```

Waveform dumping was enabled using FSDB/VCD format, and simulation results were analyzed in Verdi:

```
verdi -sv rtl/ALU/*.v tb/ALU_tb -ssf alu.vcd.fsdb &
```

6.4. Waveform Analysis (Verdi)

Waveform inspection was performed to verify:

- Correct operand registration
- Proper opcode decoding
- Accurate arithmetic results
- Correct assertion of carry flag
- Zero flag activation when result equals zero
- One-cycle latency behavior due to pipelining

The Verdi waveform confirmed that the ALU operates correctly for all tested input combinations.

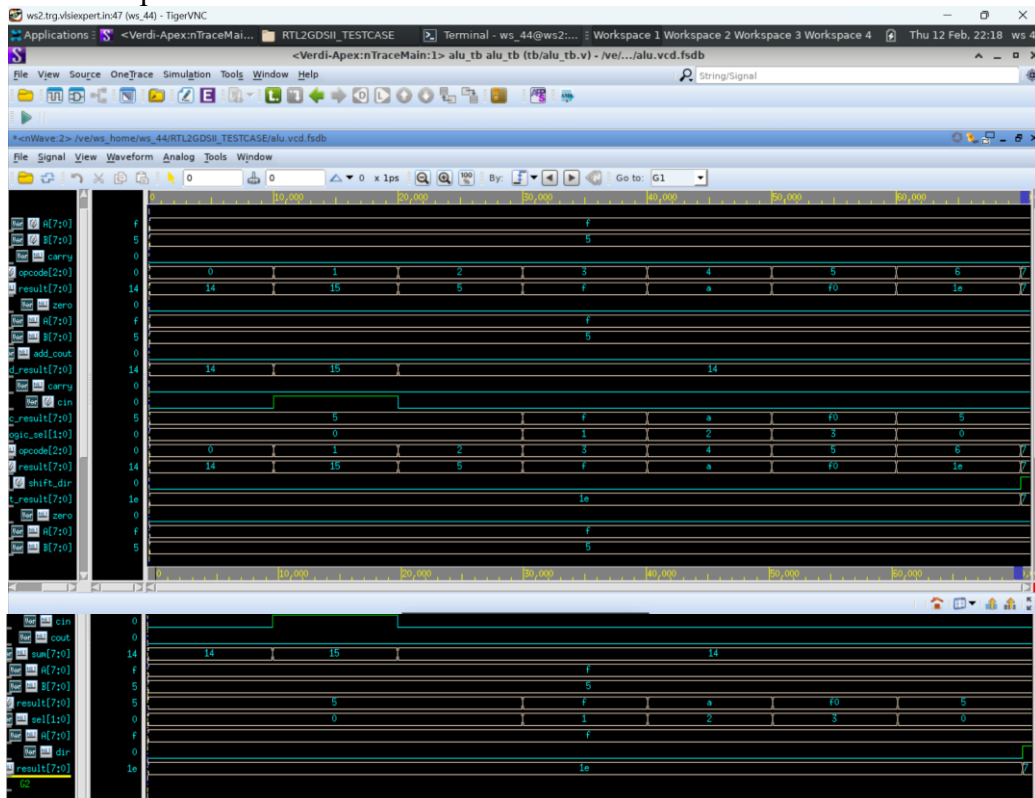


Fig. 6.1 – Verdi Waveform Showing ALU Operation and Flag Behavior

6.5. Verification Observations

- All arithmetic operations produced correct results.
- Logical and shift operations behaved as expected.
- Carry flag was correctly asserted during overflow conditions.
- Zero flag was asserted when the output result equalled zero.
- No functional mismatches were observed.
- The design successfully passed functional verification and was ready for synthesis.

7. Logic Synthesis using Synopsys Design Compiler (DC)

7.1. Introduction to Synthesis

Logic synthesis is the process of converting Register Transfer Level (RTL) code into a gate-level netlist using standard cell libraries. In this stage, the behavioural Verilog description of the 8-bit ALU is mapped to technology-specific cells from the SAED 32nm standard cell library.

The synthesis was performed using Synopsys Design Compiler in timing-driven mode with appropriate design constraints.

7.2. Synthesis Flow Overview

The synthesis process included the following steps:

1. Library setup and linking
2. RTL analysis and elaboration
3. Constraint definition (SDC)
4. Timing-driven optimization using `compile_ultra`
5. Generation of timing and QoR reports
6. Netlist generation for physical design

7.3. Design Compiler Commands Used

The following essential commands were used during synthesis:

```
dc_shell
```

```
source run_dc_alu.tcl
```

```
source -echo -verbose ./rm_setup/dc_setup.tcl
set DESIGN_NAME alu_top
set DCRM_FINAL_VERILOG_OUTPUT_FILE alu_top.mapped.v

set RTL_SOURCE_FILES "
  ./rtl/ALU/adder_8bit.v
  ./rtl/ALU/logic_unit.v
  ./rtl/ALU/shifter.v
  ./rtl/ALU/alu_core.v
  ./rtl/ALU/alu_top.v
"

define_design_lib WORK -path ./WORK

analyze -format verilog ${RTL_SOURCE_FILES}
elaborate ${DESIGN_NAME}
current_design ${DESIGN_NAME}

read_sdc ../CONSTRAINTS/alu_top.sdc

compile

write -format verilog -hierarchy -output ${RESULTS_DIR}/${DCRM_FINAL_VERILOG_OUTPUT_FILE}
```

7.4. Synthesis Results

After compilation, the RTL was successfully mapped to standard cells from the SAED 32nm library. The design was synthesized in timing-driven mode to meet clock constraints.

The synthesis process inferred:

- Flip-flops for registered input and output stages
- Combinational logic for arithmetic, logic, and shift units
- Multiplexing logic for opcode-based operation selection

7.5. Generated Schematics

The synthesized design schematics were inspected in Design Vision. The following hierarchical schematics were verified:

- Top-level module (alu_top)
- ALU core (alu_core)

- Arithmetic unit (adder_8bit)
- Logic unit (logic_unit)
- Shifter module (shifter)

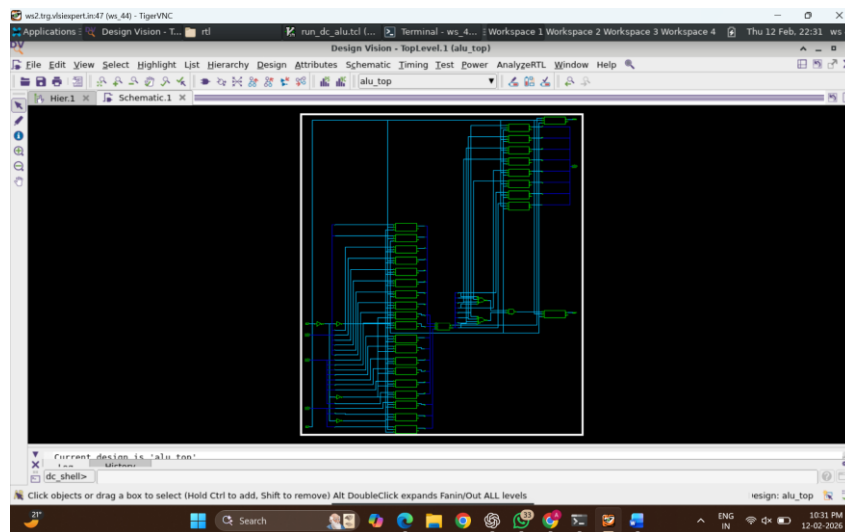
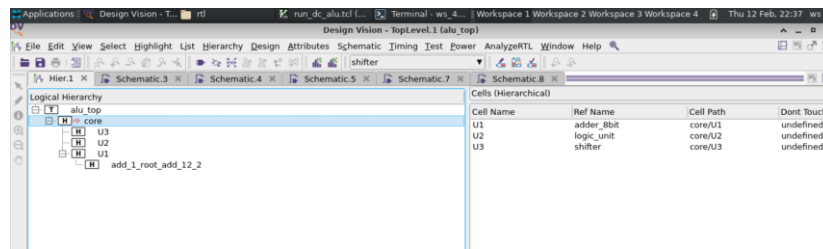


Figure 7.1 – Synthesized Top-Level ALU Schematic

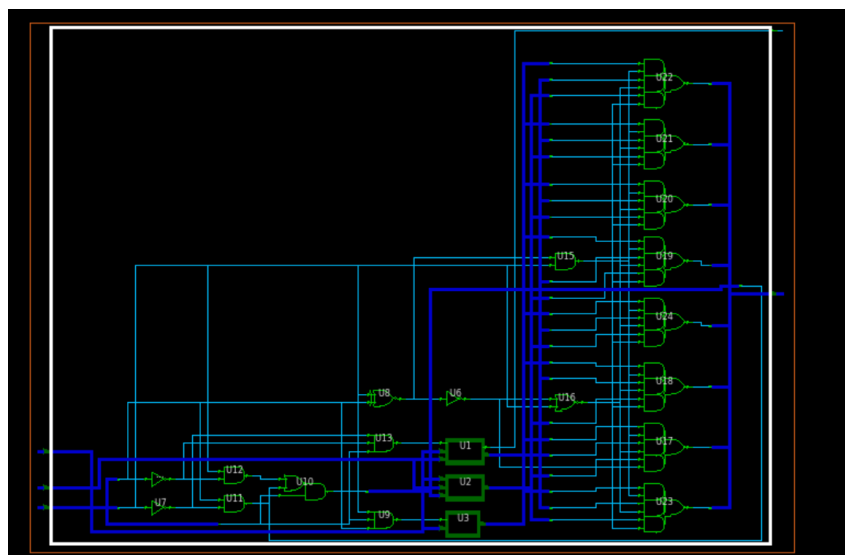


Figure 7.2 – Synthesized ALU Core Schematic

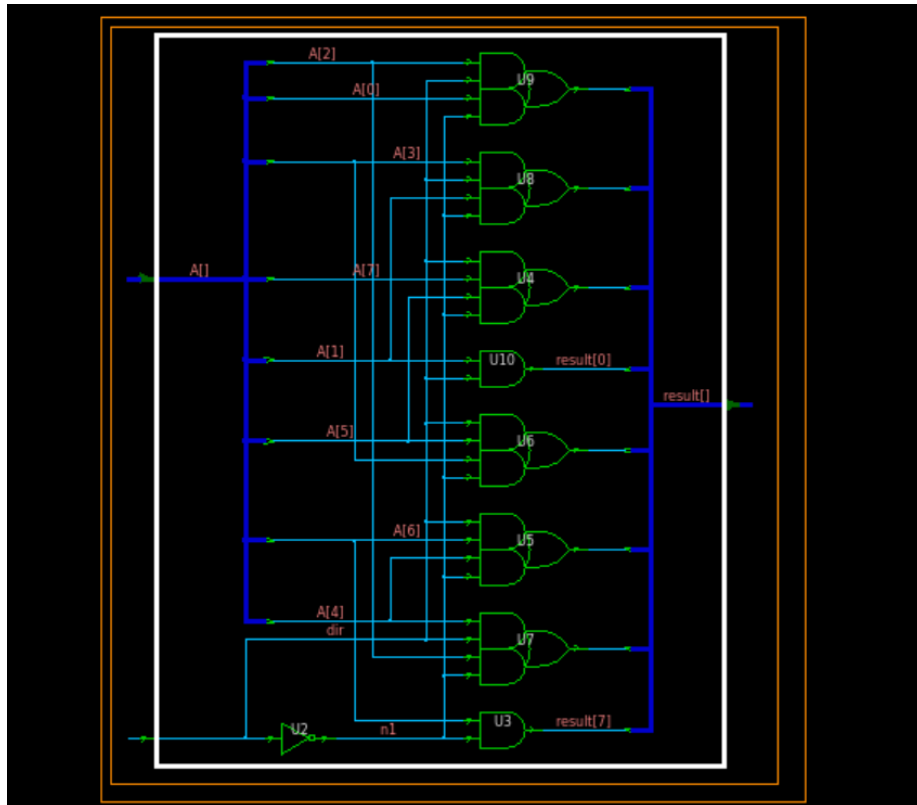


Fig 7.3 – Synthesized ALU Shifter Schematic

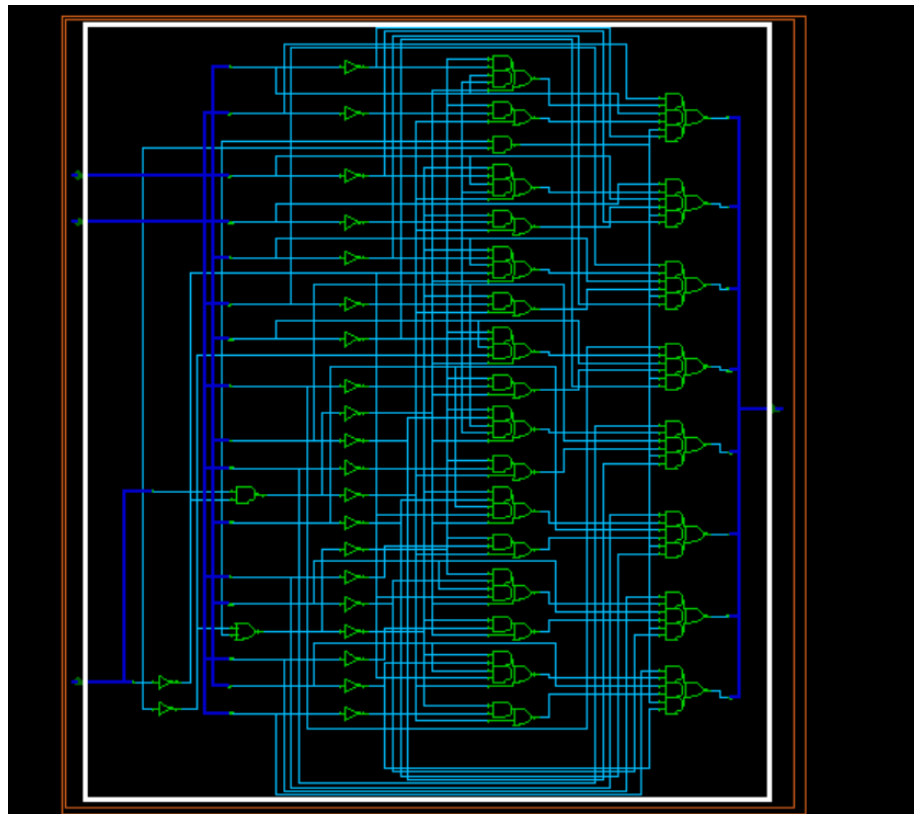


Fig 7.4. – Synthesized ALU Logic Unit Schematic

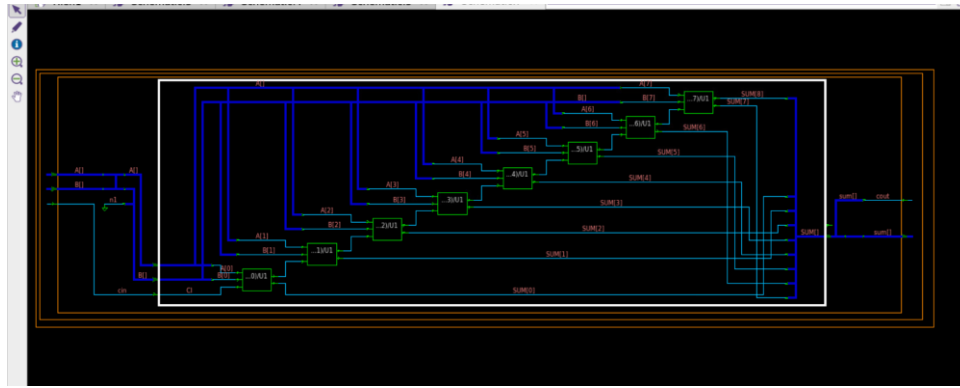


Fig 7.5. – Synthesized ALU 8 bit Adder Unit Schematic

7.6. Timing and QoR Analysis

Post-synthesis reports were analyzed to evaluate:

- Worst setup slack
- Critical path
- Total cell area
- Combinational vs sequential cell count

The arithmetic path was observed to contribute significantly to the critical timing path. Registered input and output stages helped in controlling combinational depth and improving timing closure feasibility.

The design successfully met the defined timing constraints at the synthesis stage.

The design was synthesized with a clock period constraint of **10 ns**, corresponding to a target frequency of 100 MHz. Post-synthesis timing analysis confirmed that the design met the defined setup timing requirements under the specified operating conditions. The worst-case path was observed within the arithmetic logic of the ALU core, which is expected due to the combinational depth of addition operations.

```
Current design is 'alu_top'.
dc_shell> dc_shell> report_qor
Information: Updating design information... (UID-85)
*****
Report : qor
Design : alu_top
Version: V-2023.12-SP4
Date   : Thu Feb 12 22:38:27 2026
*****

Timing Path Group 'clk'
-----
Levels of Logic:          6.00
Critical Path Length:     4.29
Critical Path Slack:      5.69
Critical Path Clk Period: 10.00
Total Negative Slack:     0.00
No. of Violating Paths:   0.00
Worst Hold Violation:     0.00
Total Hold Violation:     0.00
No. of Hold Violations:   0.00
-----
```

```
Cell Count
-----
Hierarchical Cell Count:  5
Hierarchical Port Count: 126
Leaf Cell Count:          121
Buf/Inv Cell Count:       30
Buf Cell Count:           3
Inv Cell Count:           27
CT Buf/Inv Cell Count:    0
Combinational Cell Count: 92
Sequential Cell Count:    29
Macro Count:              0
-----

Area
-----
Combinational Area:       229.746177
Noncombinational Area:    286.364932
Buf/Inv Area:             40.488896
Total Buffer Area:         6.10
Total Inverter Area:       34.31
Macro/Black Box Area:     0.000000
Net Area:                  74.239111
-----
Cell Area:                 436.111109
Design Area:               510.350219
```

```

Design Rules
-----
Total Number of Nets:      151
Nets With Violations:      0
Max Trans Violations:      0
Max Cap Violations:        0
-----

Hostname: ws2.trg.vlsiexpert.in

Compile CPU Statistics
-----
Resource Sharing:          0.05
Logic Optimization:        0.06
Mapping Optimization:      0.10
-----
Overall Compile Time:      0.77
Overall Compile Wall Clock Time: 1.02
-----

Design WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0

Design (Hold) WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0
-----

```

Fig 7.6 – report_qor

7.7. Observations

- No inferred latches were observed.
- All modules were successfully mapped to technology-specific standard cells.
- Timing-driven optimization balanced area and performance.
- The modular RTL structure improved synthesis readability and hierarchy preservation.

The synthesized gate-level netlist was then forwarded to the physical design stage using IC Compiler II.

8. Floorplanning using ICCII (IC Compiler)

8.1. Introduction

Floorplanning is the first stage of physical design where the core area, input/output pin placement, and physical boundaries of the design are defined. Proper floorplanning directly influences placement quality, routing congestion, and overall timing closure.

The synthesized netlist from Design Compiler was imported into IC Compiler II for backend implementation using the SAED 32nm technology library.

8.2. Library and Design Setup

The physical design library was created and linked with the reference technology files. The synthesized netlist was read into ICC2 and the top-level design (alu_top) was initialized for floorplanning.

8.3. Floorplan Initialization

The floorplan was initialized with a **core utilization factor (UF) of 0.65**, meaning 65% of the core area was allocated for standard cell placement while 35% was reserved for routing resources.

This utilization value was selected to:

- Avoid routing congestion
- Provide sufficient whitespace for optimization
- Improve timing-driven placement
- Ensure smooth clock tree insertion

The core offset was defined to create spacing between the core boundary and die boundary for power ring implementation. IO pins were placed along the boundary to ensure structured routing channels.

8.4. Floorplan Commands Used

The essential ICC2 commands used during floorplanning were:

```
set PDK_PATH ../../ref
create_lib -ref_lib $PDK_PATH/lib/ndm/saed32rvt_c.ndm ALU_LIB
read_verilog {../../DC/results/alu_top.mapped.v} -library ALU_LIB -design alu_top
-top alu_top
```

```
initialize_floorplan -shape U -core_utilization 0.65 -core_offset {3 3} -coincident_boundary true
set_individual_pin_constraints -ports [get_ports] -sides 1 -ports [get_ports{B}] -sides 6 -ports
[get_ports {opcode}] -sides 4 -ports [get_ports {clk rst}] -sides 1 -ports [get_ports {result z
ero}] -sides 7 -ports [get_ports {carry}] -sides 8 -pin_spacing_distance 3
place_pins -self
create_placement -floorplan -effort high
```

8.5. Floorplan Results

The generated floorplan defined:

- Core boundary
- Placement area
- IO pin distribution
- Standard cell region

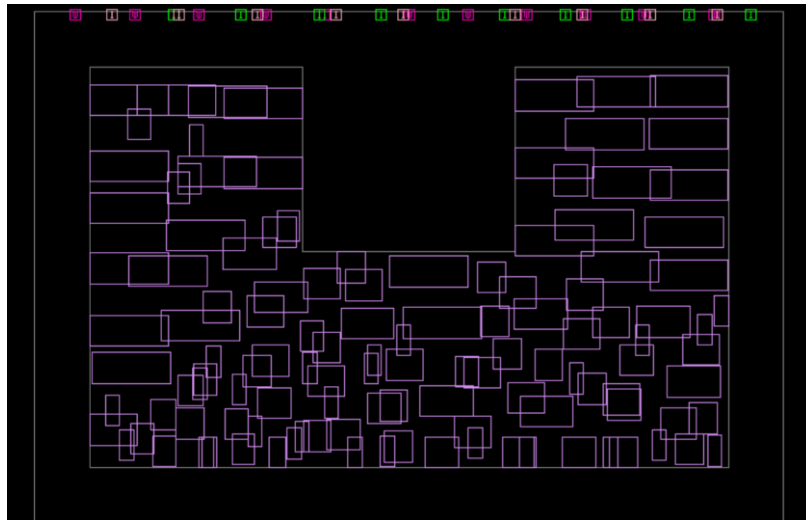


Fig 8.1. – ALU Floorplanning (U shaped)

The design shows a compact core area suitable for the small-scale ALU architecture. Proper pin placement facilitates structured routing in later stages.

8.6. Observations

- Core utilization of 65% provided sufficient placement density without causing early congestion.
- Adequate whitespace was available for routing optimization.
- The design was ready for power planning and placement stages.

9. Power Planning

9.1. Introduction

Power planning is a critical step in physical design that ensures reliable power distribution across the chip. A proper Power Distribution Network (PDN) supplies stable VDD and VSS to all standard cells while minimizing IR drop and electromigration issues.

In this project, power planning was performed in IC Compiler II after floorplan initialization.

9.2. Power Network Structure

The power distribution network was implemented using:

- Core power ring around the design
- Power mesh across higher metal layers
- Standard cell power rails in lower metal layers

This hierarchical structure ensures uniform voltage delivery across the core region.

9.3. Power Planning Commands Used

The following essential ICC2 commands were used:

```
create_net -power {VDD}
create_net -ground {VSS}
connect_pg_net -all_blocks -automatic

create_pg_ring_pattern core_ring_pattern -horizontal_layer M7 -
horizontal_width .4 -horizontal_spacing .3 -vertical_layer M8 -
vertical_width .4 -vertical_spacing .3

set_pg_strategy core_power_ring -core -pattern {{name :
core_ring_pattern}}{nets : {VDD VSS}}{offset : {.5 .5}}

compile_pg -strategies core_power_ring

create_pg_mesh_pattern mesh -layers { {{vertical_layer: M6}}{width:
.34} {spacing: interleaving}{pitch: 5} {offset: .5}}
{{horizontal_layer: M7}}{width: .38} {spacing: interleaving} {pitch:
5} {offset: .5}} {{vertical_layer: M8}}{width: .38} {spacing:
interleaving}{pitch: 5} {offset: .5}}}

set_pg_strategy core_mesh -pattern { {pattern:mesh} {nets: VDD VSS}}
-core -extension {stop: innermost_ring}

compile_pg -strategies core_mesh

create_pg_std_cell_conn_pattern std_cell_rail -layers {M1} -
rail_width 0.09

set_pg_strategy rail_strat -core -pattern {{name: std_cell_rail}
{nets: VDD VSS} }

compile_pg -strategies rail_strat

save_block

save_lib
```

9.4. Power Planning Results

The implemented power network provided:

- Stable VDD and VSS distribution
- Proper connectivity to all standard cells
- Sufficient metal width for current handling

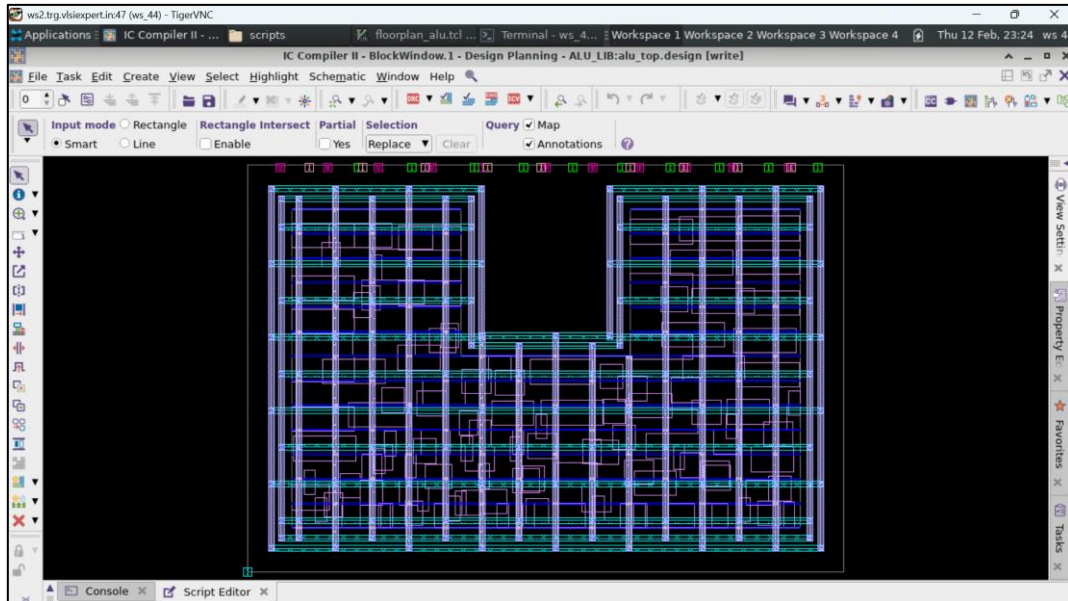


Fig 9.1. ALU Power planning

9.5. Observations

- Power and ground nets were successfully connected across the design.
 - No power connectivity violations were observed.
 - The PDN structure was suitable for the small-scale 8-bit ALU design.
- After successful power planning, the design proceeded to the placement stage.

10. Placement

10.1. Introduction

Placement is the stage of physical design where standard cells are arranged within the defined core region after floorplanning and power planning. The objective of placement is to position cells optimally to minimize wirelength, reduce congestion, and satisfy timing constraints.

For the 8-bit ALU, placement was performed in timing-driven mode using IC Compiler II.

10.2. Placement Setup

The design was configured with a single functional mode and nominal corner for timing analysis. The 10 ns clock constraint defined during synthesis was preserved to guide timing-driven placement optimization.

Parasitic technology files (TLU+) were also loaded to enable accurate RC estimation during placement.

10.3. Placement Commands Used

```

current_mode
current_corner
current_scenario

remove_modes -all; remove_corners -all; remove_scenarios -all

set mode1 "func"
set corner1 "nom"
set scenario1 "${mode1}::${corner1}"

create_mode $mode1

create_corner $corner1

create_scenario -name func::nom -mode func -corner nom

source ../../CONSTRAINTS/alu_top.sdc

current_mode func

current_corner nom

current_scenario func::nom

set parasitic1 "p1"

set                                     tluplus_file$parasitic1
"$PDK_PATH/tech/star_rcxt/saed32nm_1p9m_Cmax.tluplus"

set                                     layer_map_file$parasitic1
"$PDK_PATH/tech/star_rcxt/saed32nm_tf_itf_tluplus.map"

set parasitic2 "p2"

set                                     tluplus_file$parasitic2
"$PDK_PATH/tech/star_rcxt/saed32nm_1p9m_Cmin.tluplus"

set                                     layer_map_file$parasitic2
"$PDK_PATH/tech/star_rcxt/saed32nm_tf_itf_tluplus.map"

read_parasitic_tech -tlup $tluplus_filep1 -layermap $layer_map_filep1
-name p1

read_parasitic_tech -tlup $tluplus_filep2 -layermap $layer_map_filep2
-name p2

set_parasitic_parameters    -late_spec    $parasitic1    -early_spec
$parasitic2

set_app_options -name place.coarse.continue_on_missing_scandef -value
true

place_pins -self

```

```
place_opt
```

```
save_block -as alu_placement
```

```
save_lib
```

10.4. Placement Results

During placement:

- Standard cells were distributed uniformly within the core area.
- Timing-driven optimization adjusted cell locations to reduce critical path delay.
- No major congestion hotspots were observed.
- Placement legalization ensured no cell overlap or design rule violations.

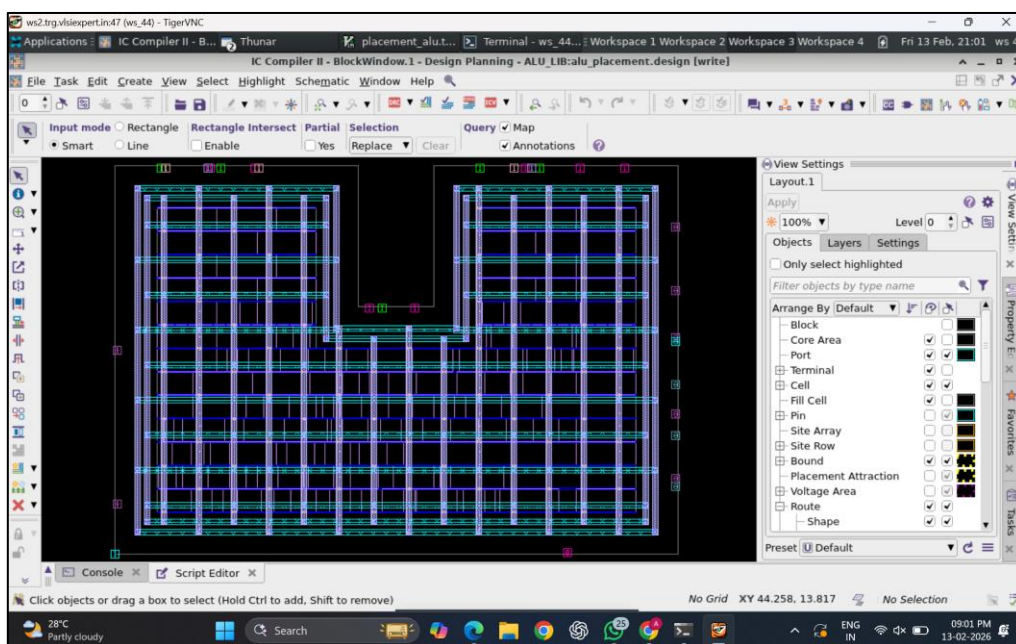


Fig. 10.1. ALU Placement

10.5. Observations

- Core utilization of 65% provided sufficient whitespace for placement optimization.
- Timing-driven placement helped maintain positive slack margins.
- The modular structure of the ALU resulted in a compact and organized cell distribution.

The design was successfully placed and prepared for Clock Tree Synthesis (CTS).

11. Clock Tree Synthesis

11.1. Introduction

Clock Tree Synthesis (CTS) is a critical stage in physical design where the clock distribution network is constructed to deliver the clock signal to all sequential elements with minimal skew and controlled insertion delay.

Before CTS, the clock is treated as ideal (zero delay). During CTS, buffers and inverters are inserted to create a balanced clock tree, ensuring proper synchronization across all flip-flops in the design.

For the 8-bit ALU, CTS was performed in timing-driven mode using IC Compiler II.

11.2. Objectives of CTS

The main objectives of CTS are:

- Minimize clock skew
- Control clock insertion delay
- Maintain setup and hold timing
- Optimize clock power consumption

Since the ALU is a synchronous pipelined design with registered inputs and outputs, proper clock balancing is essential to maintain timing integrity.

11.3. CTS Commands Used

```
#####
##      CTS using CCD  commands
#####
##
##stage 1:
##

synthesize_clock_tree
##stage 2:
##
set_app_options -name cts.optimize.enable_local_skew -value true
set_app_options -name cts.compile.enable_local_skew -value true
set_app_options -name cts.compile.enable_global_route -value false
set_app_options -name clock_opt.flow.enable_ccd -value true

#stage 3:
###
#
clock_opt
save_block -as alu_cts_CCD
save_lib
```

11.4. CTS Results

During CTS:

- Clock buffers were inserted automatically.

- Clock tree was balanced to reduce skew.
- Setup and hold timing were re-evaluated after clock insertion.
- Clock latency and insertion delay were optimized within constraints.

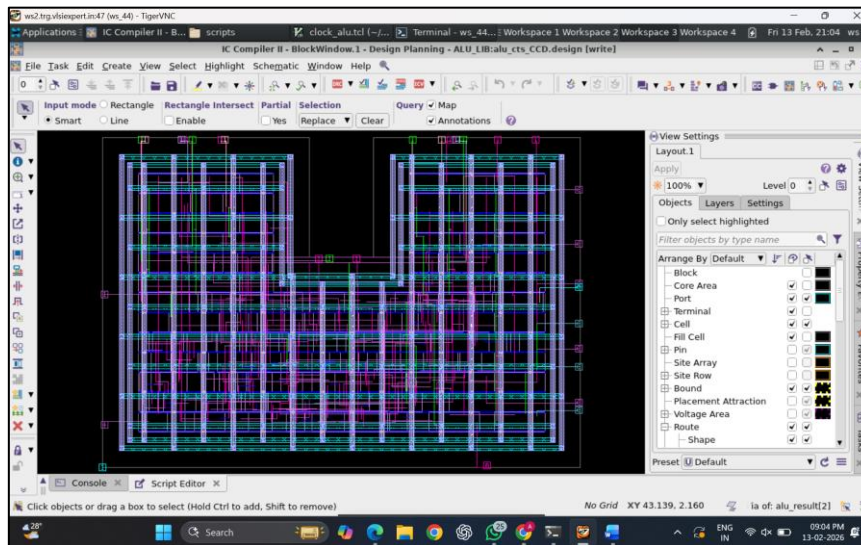


Fig. 11.1 ALU Clock Tree Synthesis

11.5. Observations

- Clock skew was minimized across all sequential elements.
- No major hold violations were introduced after CTS.
- The 10 ns clock constraint was maintained successfully.
- The design remained timing-clean after clock tree optimization.

After successful CTS, the design proceeded to the routing stage for final physical interconnections.

12. Routing

12.1. Introduction

Routing is the stage of physical design where electrical connections are created between placed standard cells according to the gate-level netlist. Metal layers and vias are used to establish interconnections while satisfying design rule constraints.

Routing is typically divided into two phases:

- **Global Routing** – Determines approximate routing paths
- **Detailed Routing** – Assigns exact metal tracks and vias

For the 8-bit ALU, routing was performed in timing-driven mode using IC Compiler II.

12.2. Routing Objectives

The primary objectives during routing were:

- Complete all signal interconnections
- Minimize wirelength
- Avoid congestion
- Meet setup and hold timing requirements
- Eliminate DRC violations
- Handle antenna effects

12.3. Routing Commands Used

```
set_app_options -name route.global.timing_driven -value true

set_app_options -name route.global.crosstalk_driven -value false

set_app_options -name route.track.timing_driven -value true

set_app_options -name route.track.crosstalk_driven -value true


set_app_options -name route.detail.timing_driven -value true

set_app_options -name route.detail.save_after_iterations -value false

set_app_options -name route.detail.force_max_number_iterations -value
false

set_app_options -name route.detail.antenna -value true

set_app_options -name route.detail.antenna_fixing_preference -value
use_diodes

set_app_options -name route.detail.diode_libcell_names -value
*/ANTENNA_RVT

route_global

route_track

route_detail

route_auto

route_opt

write_verilog ./results/alu_top.routed.v

write_sdc -output ./results/alu_top.routed.sdc
```

```

write_parasitics          -format          spef          -output
./results/alu_top_${scenario1}.spef

write_gds      -merge_files      "$PDK_PATH/lib/gds/saed32nm_rvt_oa.gds"
alu_top.gds

```

12.4. Results

During routing:

- All signal nets were successfully connected.
- Timing-driven optimization preserved setup timing margins.
- Antenna violations were automatically corrected where necessary.
- No Design Rule Check (DRC) violations were observed.

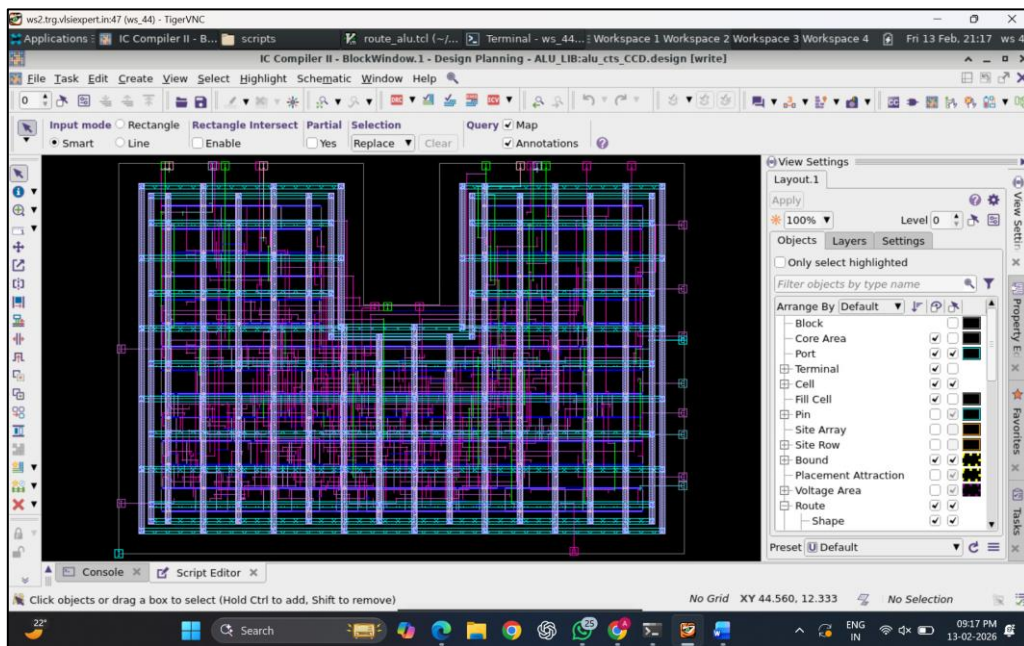


Fig. 12.1 ALU Routing

<pre> Scenario 'func::nom' Timing Path Group 'clk' ----- Levels of Logic: 15 Critical Path Length: 0.79 Critical Path Slack: 9.19 Critical Path Clk Period: 10.00 Total Negative Slack: 0.00 No. of Violating Paths: 0 Worst Hold Violation: 0.00 Total Hold Violation: 0.00 No. of Hold Violations: 0 ----- Cell Count ----- Hierarchical Cell Count: 5 Hierarchical Port Count: 144 Leaf Cell Count: 127 Buf/Inv Cell Count: 36 Buf Cell Count: 1 Inv Cell Count: 35 Combinational Cell Count: 98 Single-bit Isolation Cell Count: 0 Multi-bit Isolation Cell Count: 0 Isolation Cell Banking Ratio: 0.00% Single-bit Level Shifter Cell Count: 0 Multi-bit Level Shifter Cell Count: 0 Level Shifter Cell Banking Ratio: 0.00% Single-bit ELS Cell Count: 0 </pre>		<pre> Report : qor Design : alu_top Version: V-2023.12-SP4 Date : Fri Feb 13 21:19:50 2026 ----- Scenario 'func::nom' Timing Path Group '**in2reg_default**' ----- Levels of Logic: 0 Critical Path Length: 2.00 Critical Path Slack: 7.98 Critical Path Clk Period: 10.00 Total Negative Slack: 0.00 No. of Violating Paths: 0 Worst Hold Violation: 0.00 Total Hold Violation: 0.00 No. of Hold Violations: 0 ----- Scenario 'func::nom' Timing Path Group '**reg2out_default**' ----- Levels of Logic: 0 Critical Path Length: 0.08 Critical Path Slack: 7.92 Critical Path Clk Period: 10.00 Total Negative Slack: 0.00 No. of Violating Paths: 0 Worst Hold Violation: 0.00 Total Hold Violation: 0.00 No. of Hold Violations: 0 ----- </pre>	
--	--	--	--

12.5. Post Route Observations

- The design achieved complete routing with zero DRC violations.
- Wirelength and via count remained within reasonable limits for the design size.
- Setup and hold timing were re-evaluated after routing to confirm timing closure.
- The modular ALU structure resulted in compact and efficient routing distribution.

After successful routing, the design proceeded to post-route timing analysis and signoff verification using PrimeTime.

13. GDSII Generation

13.1. Introduction

After successful routing and verification of Design Rule Checks (DRC), the final layout database was generated in GDSII format.

The GDSII file represents the physical layout of the chip and is the final output of the RTL-to-GDSII flow, ready for fabrication.

13.2. GDSII Generation Process

Once routing was completed and the design was verified to be DRC-clean, the GDS file was exported using IC Compiler II.

The following command was used to generate the GDSII file:

```
write_gds alu_top.gds
```

This command converts the routed layout database into the industry-standard GDSII format.

13.3. GDS Verification

Before final export, the following checks were ensured:

- No DRC violations
- Proper power and ground connectivity
- Fully routed signal nets
- Clock tree correctly implemented
- Timing closure achieved

13.4. Observations

- The design was successfully converted to GDSII format.
- No physical rule violations were present.

- The final layout was compact and well-structured.
- The RTL-to-GDSII implementation flow was completed successfully.

The generated GDSII file represents the manufacturable physical implementation of the 8-bit ALU.

14. Static Timing Analysis Using PrimeTime

14.1. Introduction

Static Timing Analysis (STA) is the final verification step in the digital design flow used to validate timing performance without applying simulation vectors. It ensures that all timing paths meet setup and hold requirements under defined operating conditions.

For the 8-bit ALU, post-route timing analysis was performed using Synopsys PrimeTime after GDS generation.

14.2. STA Objectives

The main objectives of STA were:

- Verify setup timing constraints
- Verify hold timing constraints
- Identify critical timing paths
- Confirm positive slack
- Ensure timing closure at the defined clock period

The analysis was performed using a clock constraint of **10 ns (100 MHz)**.

14.3. PrimeTime Commands Used

```
set report_default_significant_digits 6

set link_path "*" ../../ref/lib/stdcell_rvt/saed32rvt_ss0p7vn40c.db"

read_verilog "../../ICII/results/alu_top.routed.v"

link_design alu_top

current_design alu_top

read_sdc ../../CONSTRAINTS/alu_top.sdc

read_parasitics
"../../ICII/results/alu_top_func::nom.spef.p1_125.spef"

set_propagated_clock [all_clocks]

update_timing -full
```

Reports

```
report_timing -delay_type max -max_paths 5
```

```
report_timing -delay_type min -max_paths 5
```

```
report_design
```

```
check_timing -verbose > ./reports/check_timing/check_timing.p1_report
```

```
report_global_timing >
./reports/timing/report_global_timing.p1_report
```

```
report_clock -skew -attribute >
./reports/clock/report_clock.p1_report
```

```
report_analysis_coverage >
./reports/analysis_coverage/report_analysis_coverage.p1_report
```

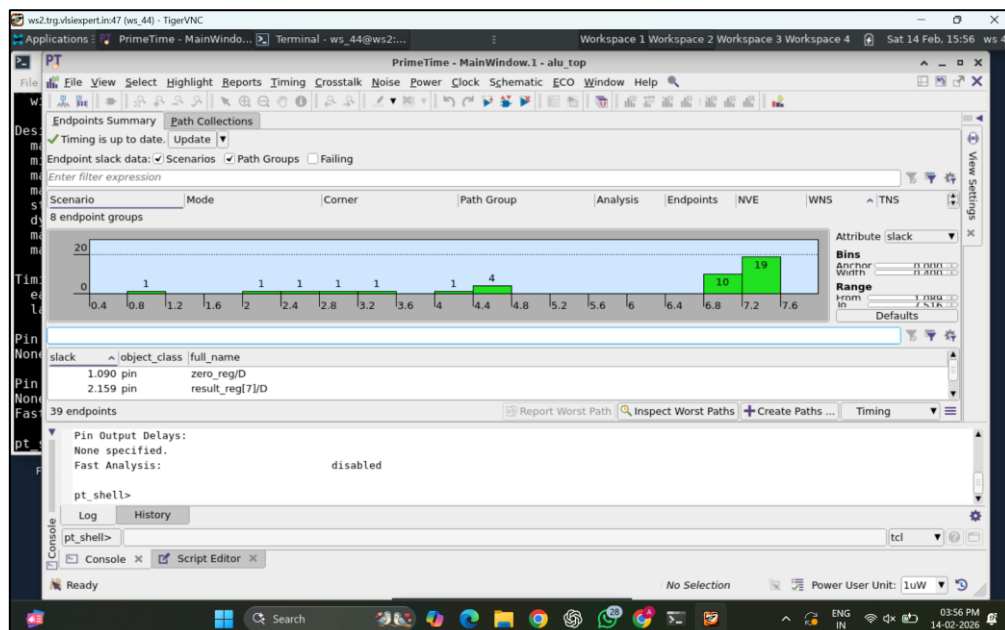
```
report_timing -slack_lesser_than 0.0 -delay min_max -nosplit -input -
net > ./reports/timing/report_timing.p1_report
```

14.4. Setup Timing Analysis

Setup timing ensures that data arrives at the destination flip-flop before the required clock edge.

Post-route setup timing analysis confirmed:

- Worst negative slack (WNS) ≥ 0
- All paths met the 10 ns clock constraint
- Critical path located within ALU arithmetic logic



14.5. QoR Summary

```

*****
Report : qor
Design : alu_top
Version: W-2024.09-SP1
Date   : Sat Feb 14 22:55:59 2026
*****

Timing Path Group 'clk' (max_delay/setup)
-----
Levels of Logic:                16
Critical Path Length:           8.551995
Critical Path Slack:            1.090937
Total Negative Slack:           0.000000
No. of Violating Paths:         0
-----

Timing Path Group 'clk' (min_delay/hold)
-----
Levels of Logic:                2
Critical Path Length:           1.266069
Critical Path Slack:            1.379757
Total Negative Slack:           0.000000
No. of Violating Paths:         0
-----

Area
-----
Net Interconnect area:          86.256676
Total cell area:                439.923279
Design Area:                    526.179932
-----

Cell & Pin Count
-----
Pin Count:                      537
Hierarchical Cell Count:        5
Hierarchical Port Count:       134
Leaf Cell Count:                127
-----

Design Rule Violations
-----
Total No. of Pins in Design:    537
Total DRC Cost:                 0.000000
-----
1

```

14.6. Final Observations

- The 8-bit ALU successfully met all setup and hold timing requirements.
- The pipelined architecture contributed to improved timing stability.
- The complete RTL-to-GDSII flow was validated through post-route signoff timing analysis.

The design is therefore considered timing-clean and physically implementable under the specified constraints.

15. Conclusion

The complete RTL-to-GDSII implementation of the 8-bit Arithmetic and Logic Unit (ALU) was successfully carried out using the Synopsys EDA tool flow. The design progressed through all major stages of ASIC implementation, including RTL design, functional verification, logic synthesis, floorplanning, power planning, placement, clock tree synthesis, routing, GDSII generation, and post-route static timing analysis.

The ALU was implemented using a modular and synchronous architecture with registered input and output stages, ensuring stable timing behavior and improved physical design feasibility. Functional verification using VCS and Verdi confirmed correct operation for all supported arithmetic, logical, and shift operations.

During synthesis and physical implementation, timing-driven optimization was applied with a clock constraint of 10 ns (100 MHz). Post-route analysis using PrimeTime verified that all setup and hold timing requirements were satisfied with positive slack. The final layout was DRC-clean and successfully exported in GDSII format.

Summary Table:

Parameter	Final Result
Clock Period	10 ns
Frequency	100 MHz
Worst Setup Slack	≥ 0
Hold Violations	None
DRC Violations	0
Core Utilization	65%

This project provided practical exposure to the complete ASIC design methodology and strengthened understanding of timing-driven design, clock tree balancing, and post-route signoff verification. The hands-on experience gained from implementing the 8-bit ALU through the full RTL-to-GDSII flow bridges the gap between theoretical digital design concepts and real-world semiconductor implementation practices. The successful implementation demonstrates a complete understanding of both frontend and backend ASIC design methodologies.

